

COVERING CODES CHECKED BY THE LEAN KERNEL: $K_7(9, 4) \leq 1141$, SEVEN MORE UPPER BOUNDS, AND $K_2(6, 1) = 12$

THIAGO PATZDORF

ABSTRACT. We prove in Lean 4, against Mathlib v4.34.1 and checked by the kernel with no `sorry` and no `native_decide`, upper bounds on the covering number $K_q(n, R)$ for eight cells, each by an explicit code smaller than the corresponding entry in the tables we consulted. The largest is $K_7(9, 4) \leq 1141$, against a best published bound of 1475; the others are $K_7(8, 3) \leq 1887$, $K_5(10, 4) \leq 625$, $K_5(9, 3) \leq 1250$, $K_5(7, 2) \leq 500$, $K_5(9, 4) \leq 250$, $K_4(10, 4) \leq 192$ and $K_5(9, 5) \leq 50$. The codes are unions of cosets of a linear code plus a small patch, and we give two certificates: one walks digit prefixes and costs 12 CPU-hours for $(\mathbb{Z}/7\mathbb{Z})^9$; the other works on syndromes and costs about ten CPU-minutes. The best codes were found by enumerating all 6362 equivalence classes of $[9, 3]_7$ codes. We also prove $K_2(6, 1) \geq 11$ by a short double-counting argument and the classical value $K_2(6, 1) = 12$ by a search whose soundness is a theorem.

1. INTRODUCTION

The covering number $K_q(n, R)$ is the least cardinality of a code $C \subseteq A^n$, with $|A| = q$, such that every word of A^n lies at Hamming distance at most R from some word of C [1]. Upper bounds come from constructions and lower bounds from counting; the tables of Kéri [4] and the recent work of Gijswijt and Polak [3] and of Marosi [6] record the current state.

Upper bounds are certificates: a list of words and the claim that it covers. Checking the claim is a finite computation, but for q^n in the tens of millions it is done by programs that are rarely audited. Florath [2] built a Lean library of covering-code bounds whose derivations are replayed by the kernel; it formalizes the sphere-covering bound and records many cells. This note adds certificates that the kernel checks for eight cells where the codes are new, a certificate that scales by working on syndromes, and a formal determination of a small classical value.

The proofs use Lean 4 [5] and Mathlib [7]. The source is <https://github.com/thiagopatzdorf/Matematica>, tag v0.4.0. A statement below is a claim only when it is the name of a declaration in that tree. The default target `CoveringLean` builds in seconds; the prefix certificates form the target `CoveringHeavy` and the syndrome certificates the target `CoveringSyn`.

2. DEFINITIONS

In the development `CoveringA2` a word is a function $\text{Fin } n \rightarrow \text{ZMod } q$, distance is Mathlib's `hammingDist`, and

$$\text{Covers } R \ C : \iff \forall x, \exists c \in C, d(x, c) \leq R.$$

In `CoveringA6` the alphabet is $\text{Fin } q$, with the same distance and the same notion of covering, and

$$\text{IsK } q \ n \ R \ k : \iff (\exists C, |C| = k \wedge \text{Covers } R \ C) \wedge \forall C, |C| < k \rightarrow \neg \text{Covers } R \ C,$$

which is the equality $K_q(n, R) = k$.

3. UPPER BOUNDS FOR $K_7(9, 4)$

Theorem 1. *There is $C \subseteq (\mathbb{Z}/7\mathbb{Z})^9$ with $|C| = 1141$ that covers at radius 4. In Lean: `Syn.K7_9_4_le.1141_syn`, stated as*

$$\exists C : \text{Finset}(\text{Fin } 9 \rightarrow \mathbb{Z}\text{Mod } 7), \quad C.\text{card} = 1141 \wedge \text{Covers } 4 \, C.$$

The library proves the same statement for three codes of this cell, found in succession: 1351 words (`CoveringKernel.K7_9_4_le.1351_kernel`, and again as `Syn.K7_9_4_le.1351_syn`), 1285 words (`Syn.K7_9_4_le.1285_syn`) and 1141 words. Each code is a list of natural numbers $\sum_i w_i 7^i$ in strictly increasing order, and the same list is a text file in `data/codes/`; the canonical SHA-256 of the 1141-word code begins `bcb05960`.

Structure. All three codes are three cosets $r_j + C_0$ of a linear $[9, 3]_7$ code C_0 (1029 words) together with a patch. Write C_0 as the kernel of $H = [I_6 \mid A]$; the three cosets cover exactly the points whose syndrome is within distance-4 reach of one of three syndromes, and the remaining syndromes are *orphans*, whose 343 points each must be covered by the patch. For the 1351-word code there are 27 orphans and 322 patch words; for 1285, 24 orphans and 256 patch words; for 1141, with A given by the rows 666, 065, 652, 643, 621, 615 and coset syndromes 0, 7708, 4191, there are only 6 orphans (2058 points) and 112 patch words.

First certificate: digit prefixes. A Boolean function `go` walks the coordinates one at a time, keeping for each prefix the codewords still within reach and their remaining budget of mismatches; `go_sound` proves that `go` returning `true` implies that every word has a codeword at distance at most R . `covers_of_go` turns that into `Covers`, using that Mathlib’s distance agrees with the digit-wise count, and `cert_of_go` adds the cardinality, from a kernel check that every entry is below q^n and the list is strictly increasing. `go_split` fixes a prefix of coordinates in advance, so the computation becomes many leaf theorems, each closed by `decide +kernel`; for $K_7(9, 4) \leq 1351$ there are $7^4 = 2401$ leaves in 95 files, about 12 CPU-hours in total. The pipeline accepts the binary Hamming code of length 7 and rejects a copy of it with one word replaced; both are tests in the library.

Second certificate: syndromes. The second certificate uses the structure. `SynCheck` (core Lean only, no Mathlib) defines Boolean checks over a transversal of the q^{n-k} syndrome classes: each class gets a witness (coset, message) or is marked orphan; each orphan’s q^k points are checked against the list; and each coset word $r_j + mG$ is shown to be in the list. `SynBridge` (with Mathlib, of fixed size) proves the checks sound, transports coverage along the translation by C_0 , and concludes $\exists C, C.\text{card} = M \wedge \text{Covers } R \, C$ in the theorem `syn_cert`. The leaves are `decide +kernel` over blocks of 4096 classes and import only `SynCheck`. The 1351-word code costs about 8 CPU-minutes this way instead of 12 CPU-hours, and each leaf stays below 1.8 GB. Each certificate also contains an `example` in which a wrong witness is rejected by the kernel.

Remark. The best upper bound we found is $K_7(9, 4) \leq 1475$, by Marosi [6] (arXiv version 3, September 2026); Kéri’s tables [4] give 1843. The best lower bound we know is 264. That the bound is new is our reading of those sources on 2 October 2026, not something the kernel checks.

4. A LOWER BOUND WITHOUT SEARCH: $K_2(6, 1) \geq 11$

Lemma 2. *For distinct $a, c \in (\mathbb{F}_2)^6$, the balls of radius 1 about a and c meet in 0 or 2 words.*

Proof. At distance 1 the intersection is $\{a, c\}$; at distance 2 it is the two words between them; at distance at least 3 it is empty. In Lean the case $a = 0$ is a kernel decision over 64 words (`even_inter_zero`), and translation gives the general case (`inter_card_translate`). $\square$

Theorem 3. *Every code that covers $(\mathbb{F}_2)^6$ at radius 1 has at least 11 words (`CoveringA6.K_2_6_1_ge.11`).*

Proof. Let $K = |C|$ and let $m(y)$ be the number of codewords whose ball contains y , so $m(y) \geq 1$ and $\sum_y m(y) = 7K$. For each x put $S(x) = \sum_{y \in B(x)} m(y) = \sum_{c \in C} |B(x) \cap B(c)|$. Then $S(x) \geq 7$, and if $x \notin C$ every term is even by Lemma 2, so $S(x) \geq 8$. Summing $S(x) + [x \in C] \geq 8$ over all 64 words and using $\sum_x S(x) = 7 \sum_y m(y) = 49K$ gives $50K \geq 512$, hence $K \geq 11$. The counting is `excess_bound`, stated for any family of symmetric balls of size 7 with even pairwise intersections. $\square$

This replaces a hypothesis that the previous version of this note left open: the statement `no_cover_2_6_1_1e10_uncond` is the earlier conditional one with the condition removed.

5. AN EXACT VALUE: $K_2(6, 1) = 12$

Theorem 4. `SC.K_2_6_1_eq12 : IsK 2 6 1 12.`

The value is classical [8]. The upper bound is an explicit 12-word code (`code12`, encodings 0, 1, 2, 15, 28, 23, 57, 58, 59, 39, 44, 52). For the lower bound, a search `chkN` on 64-bit masks takes a state (words still allowed, covered set, forbidden centres), picks the least uncovered word, and branches on the centres that could cover it, forbidding at each branch the siblings already tried. It prunes when the remaining words cannot cover what is left, at 7 words per centre. `chkN_sound` proves that a `true` answer means no completion exists. A translation argument lets one assume the word 63 is in the code. The search from that root is cut at depth 2 into 38 states, each refuted by `decide +kernel` in its own file. The Lean database of Florath [2] records $10 \leq K_2(6, 1) \leq 12$ at this cell.

6. THE SPHERE-COVERING BOUND

The library also proves the sphere-covering bound $q^n \leq |C| \cdot V$ (`CoveringA2.sphere_covering`), its strict form when $V \nmid q^n$, and eight numerical instances `CoveringChain.SPH.K*_lb`, among them $K_7(9, 4) \geq 221$. This was the content of version 0.2 of this note. The bound had already been formalized by Florath [2], and all eight instances are below the best lower bounds in the literature (for $K_7(9, 4)$, 264), so they are not claimed as results.

7. SEVEN MORE UPPER BOUNDS

The same certificates prove seven more upper bounds (Table 1). Each declaration is stated as $\exists C, C.\text{card} = M \wedge \text{Covers } R \ C$ over $\text{Fin } n \rightarrow \text{ZMod } q$, and for each cell the list in Lean decodes to the code file in `data/codes/` with the same canonical SHA-256. For $q = 4$ our literature search was weaker than for $q \geq 5$, and $K_5(10, 4) \leq 625$ is a linear $[10, 4]_5$ code, which should still be compared with the tables of Davydov, Marcugini and Pambianco on linear covering codes.

cell	ours	previous	source	certificate
$K_7(9, 4)$	1141	1475	Marosi	syndromes (also 1285; 1351 by prefixes)
$K_7(8, 3)$	1887	2337	Kéri	syndromes (1893 by prefixes)
$K_5(10, 4)$	625	875	Kéri	prefixes
$K_5(9, 3)$	1250	1275	Kéri	prefixes
$K_5(7, 2)$	500	525	Kéri	prefixes
$K_5(9, 4)$	250	255	Kéri	prefixes
$K_4(10, 4)$	192	208	Kéri	prefixes
$K_5(9, 5)$	50	55	Kéri	prefixes

TABLE 1. Upper bounds proved in Lean and checked by the kernel.

8. HOW THE CODES WERE FOUND

The first codes came from the generator `lincov` of Marosi’s public repository `Mapika/coldcase` [6] followed by greedy patching. The improvements came from treating the base as the main variable. For $K_7(9, 4)$ we enumerated all 6362 classes of non-degenerate $[9, 3]_7$ codes up to equivalence and, for each, computed exactly the minimum number of orphan syndromes left by three cosets; the minimum 6, against 27 for the original base, gives the 1141-word code after optimizing the patch with a simulated-annealing search on the residual points only. For $K_7(8, 3)$ the base of the 1893-word code is five cosets of an $[8, 3]_7$ code (1715 words) with two orphan syndromes (686 points); reoptimizing the patch gives 172 words, which is optimal for that base, since no ball of radius 3 contains more than four of the 686 points and $\lceil 686/4 \rceil = 172$. Every candidate was checked by programs independent of the one that found it before being certified in Lean; the codes, a structured description of each (base, cosets, patch, provenance) and an official checker are in the repository.

9. WHAT THE KERNEL ACCEPTED

On 2 October 2026, on a virtual machine of type `e2-highmem-8`, Lean 4.34.1 and Mathlib v4.34.1: `lake build` (default target, from a fresh clone) finished 8944 jobs in 4 min 16 s, peak 10.0 GB; `lake build CoveringHeavy` (the prefix certificates and $K_2(6, 1) = 12$) finished 9181 jobs in 1 h 54 min, about 9.3 CPU-hours, at most 9.4 GB per process; `lake build CoveringSyn` (the four syndrome certificates) finished: 8983 jobs in 10 min 35 s on 8 cores (37 CPU-minutes for all four), peak 6.9 GB per process. `#print axioms` on Theorems 1, 3, 4 and on every declaration of Table 1 reports `propxt`, `Classical.choice` and `Quot.sound` only.

10. WHAT IS NOT A THEOREM

Novelty is a reading of the literature, not a checked fact. The search programs are not verified; only their output is. That the formal statements say what is intended is a human review: the reader should check the statements of Theorems 1, 3, 4 and the definitions of `Covers` and `IsK`. Two files in the tree, `A5b.Stress.lean` and `A6d.SearchHeavy.lean`, are not imported and are not claimed.

REFERENCES

- [1] G. Cohen, I. Honkala, S. Litsyn and A. Lobstein, *Covering Codes*, North-Holland, 1997.
- [2] A. Florath, *Formal foundations and proof-carrying certificates for q -ary covering codes in Lean 4*, arXiv:2606.09600, 2026.
- [3] D. Gijswijt and S. Polak, *Semidefinite lower bounds for covering codes*, arXiv:2504.01932, 2025.
- [4] G. Kéri, *Tables for bounds on covering codes*, <https://old.sztaki.hu/~keri/codes/>, 2011.
- [5] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, CADE 28, LNCS **12699**, Springer, 2021, 625–635.
- [6] M. Marosi, *New upper and lower bounds on covering codes $K_q(n, R)$ for alphabets of size $5 \leq q \leq 21$* , arXiv:2608.19872, 2026.
- [7] The Mathlib Community, *Mathlib*, <https://github.com/leanprover-community/mathlib4>, release v4.34.1.
- [8] R. G. Stanton and J. G. Kalbfleisch, *Covering problems for dichotomized matchings*, Aequationes Math. **1** (1968), 94–103.

UNIVERSIDADE DE CAXIAS DO SUL, BRAZIL
 Email address: thiagosleman@gmail.com